

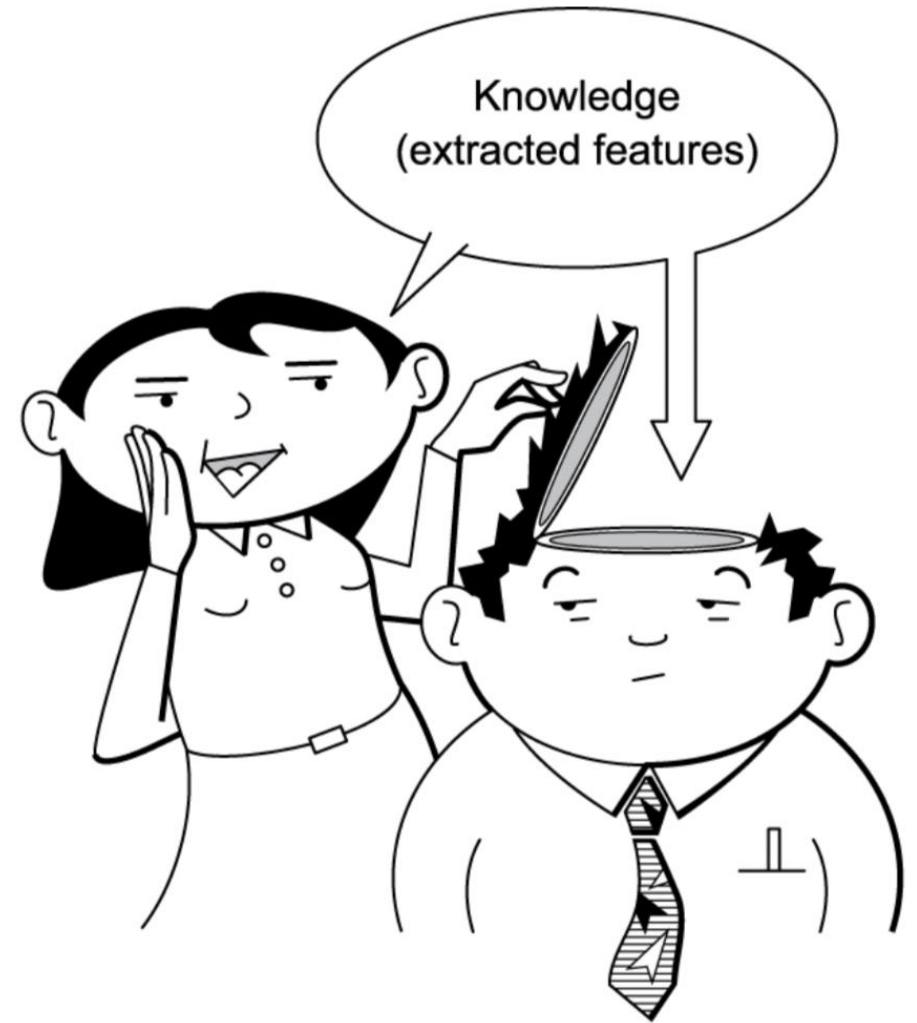
Lecture 13

Transfer Learning



Transfer Learning

- Transferring the knowledge from a pretrained model in one domain to the task in another domain
- Uses the large number of extracted features from the original dataset for a pretrained model. Trains the pretrained model with the new smaller dataset
- Enables to train the model with less data as compared to the original dataset on which the model has been pretrained
- Requires less training time



Why to use transfer learning?

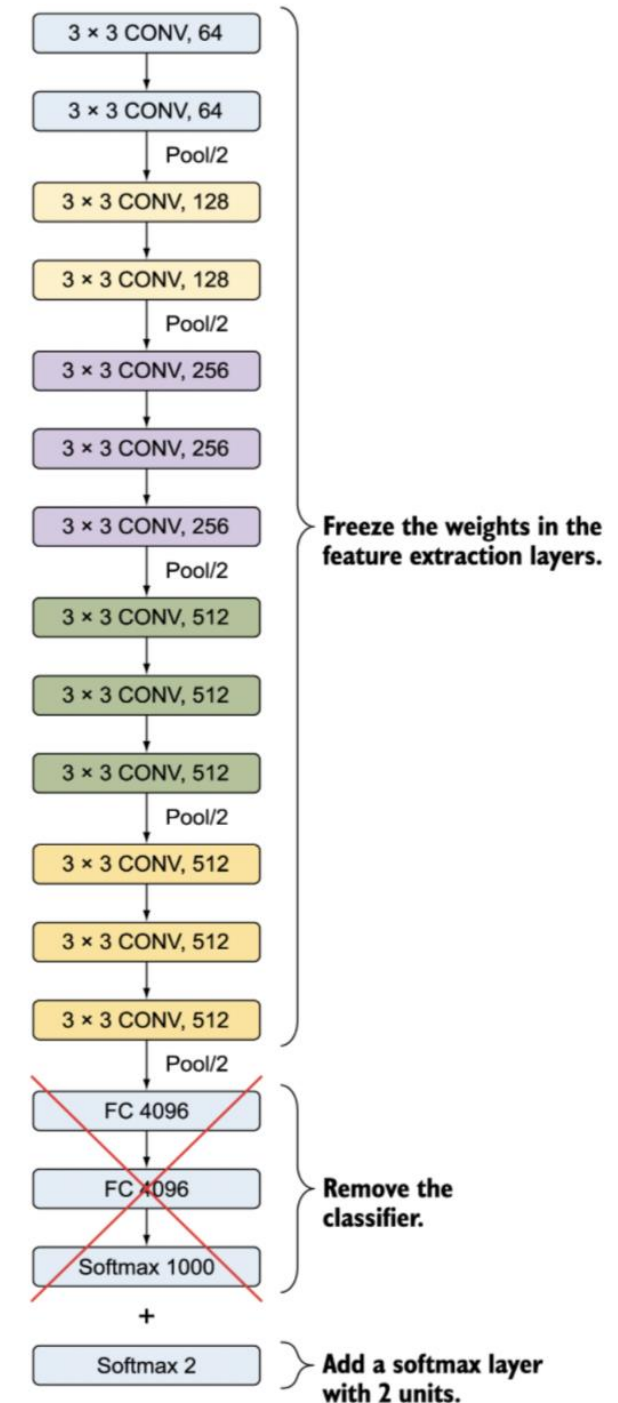
- Training data availability
 - Number of training samples
 - Number of labeled data samples
- Computation problem
 - Computationally very expensive to train large DL models
 - Required training time
- Helps with generalizing the model learning and avoids overfitting

Transfer Learning

- *“Transfer learning is the transfer of the knowledge (feature maps) that the network has acquired from one task, where we have a large amount of data, to a new task where data is not abundantly available.”*
- Even though the large labeled dataset, huge compute power and time is available, it's better to use transfer learning.
 - Sets the model on a faster track of learning
 - Helps to avoid overfitting

Example

- Pretrained model - VGG16
- Base model training dataset – ImageNet
- Used pretrained model as a feature extractor
- Removed the fully connected and classification layers
 - Keras code
 - `from keras.applications.vgg16 import VGG16`
 - `base_model = VGG16(weights = "imagenet", include_top=False, input_shape = (224,224, 3))`
 - `include_top=False` -> ignores the FC layers in the network
- Model summary
 - Total params: 14,714,688
 - Trainable params: 14,714,688
 - Non-trainable params: 0
- Layers freezing
 - `for layer in base_model.layers: layer.trainable = False`
 - `base_model.summary()`
 - Total params: 14,714,688
 - Trainable params: 0
 - Non-trainable params: 14,714,688



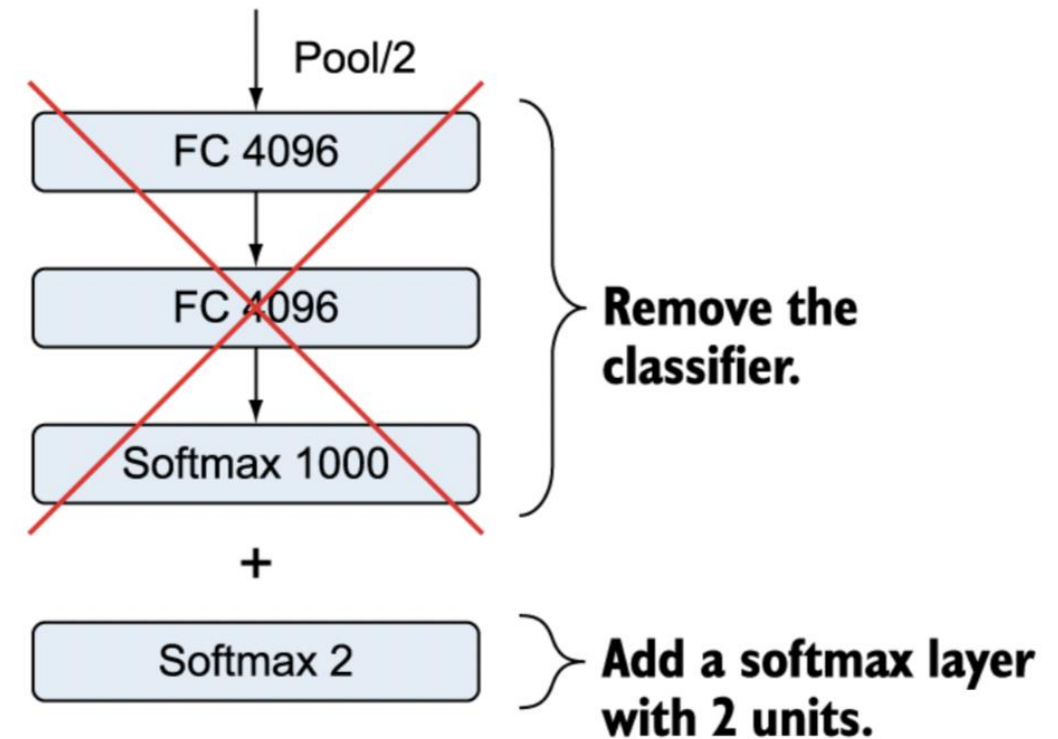
Example – cont.

- Add new classification layer

- `from keras.layers import Dense, Flatten`
- `from keras.models import Model`
-
- `last_layer = base_model.get_layer('block5_pool')`
- `last_output = last_layer.output`
-
- `x = Flatten()(last_output)`
-
- `x = Dense(2, activation='softmax', name='softmax')(x)`

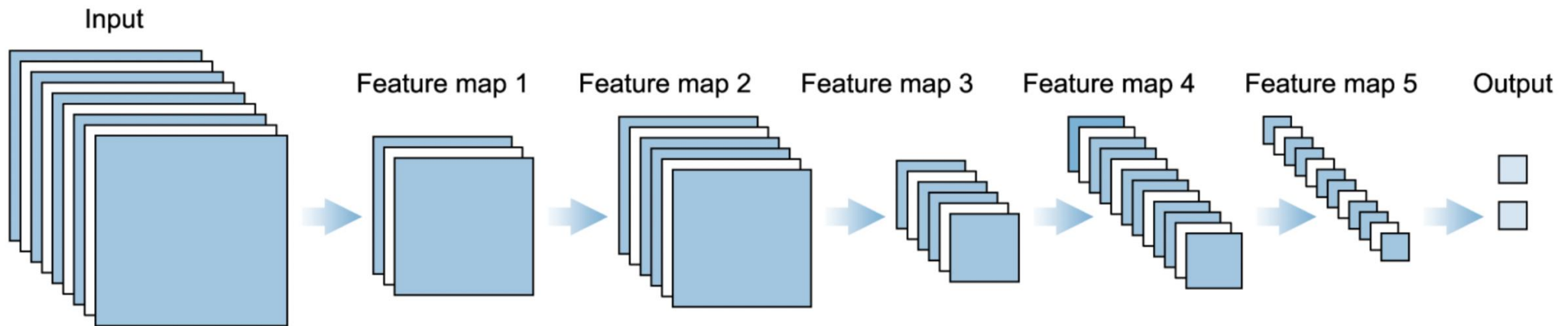
- New model summary

- Total params: 14,789,955
- Trainable params: 50,178
- Non-trainable params: 14,714,688



How Transfer Learning Works

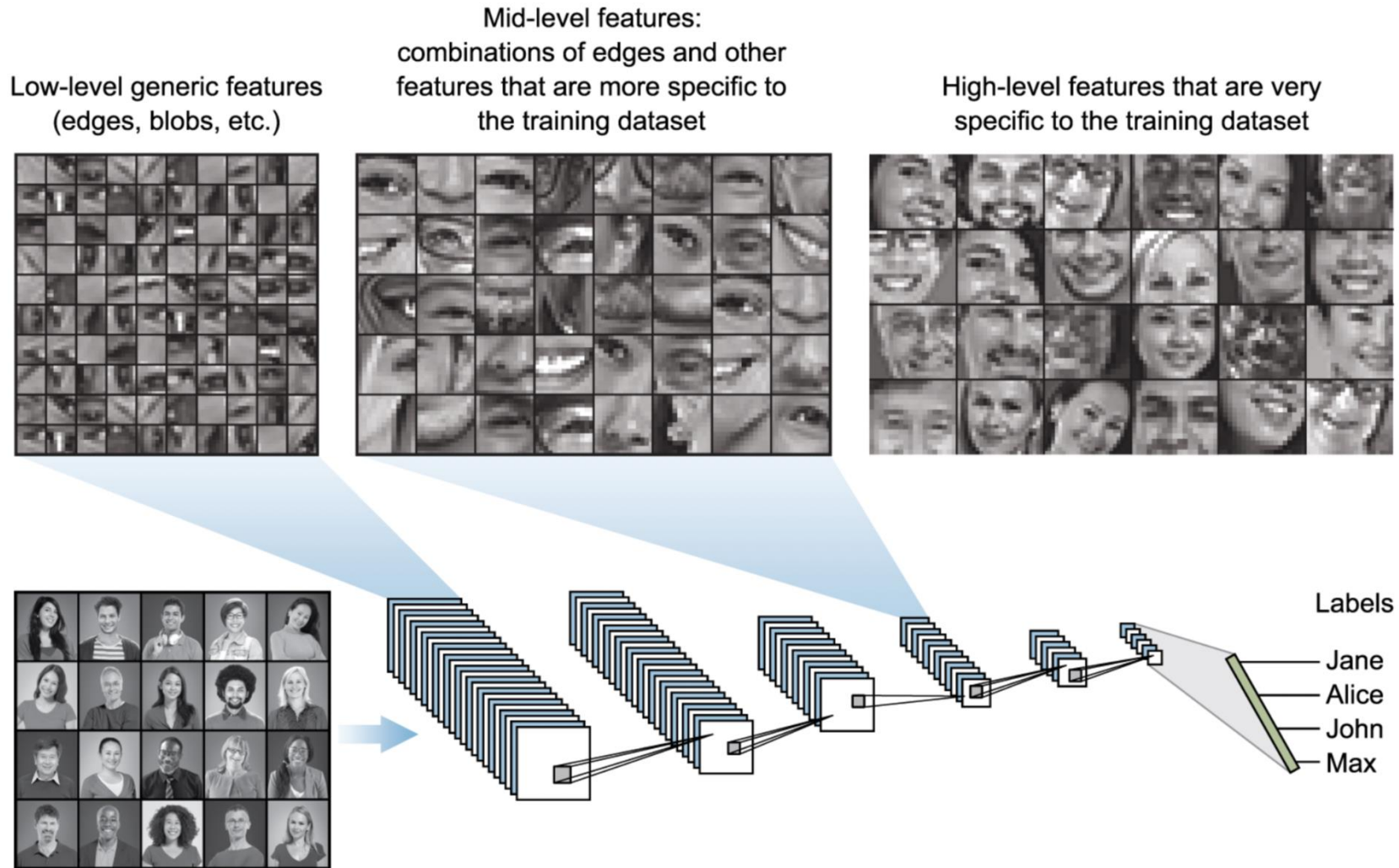
- Features maps get learned during model training
- During transfer learning, the optimized weights from the pre-trained models get transferred to new model
- Features maps are formed after applying weight filters on input of each layer



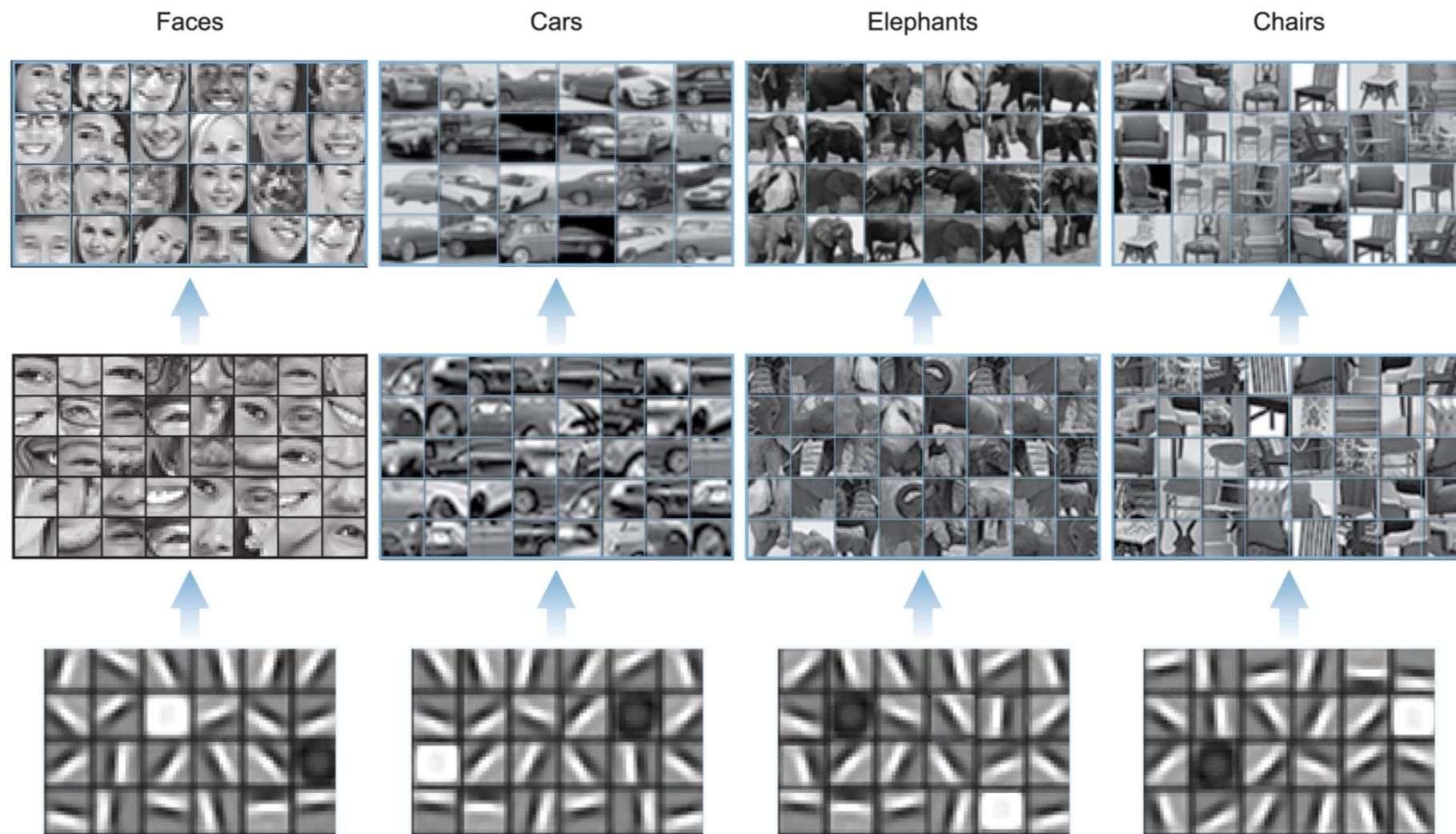
How Transfer Learning Works – cont.

- Trained model provides the model architecture and the trained weights
- The pretrained models see a huge amount of data to be generalized for different settings such as day& night, different camera resolutions, colors, background which can not be achieved with the smaller dataset in hand
- Pretrained models allow to reuse the features extracted from the large datasets and give a quick start for training the models on new dataset

Feature learning in NN



Feature maps from 4 different models

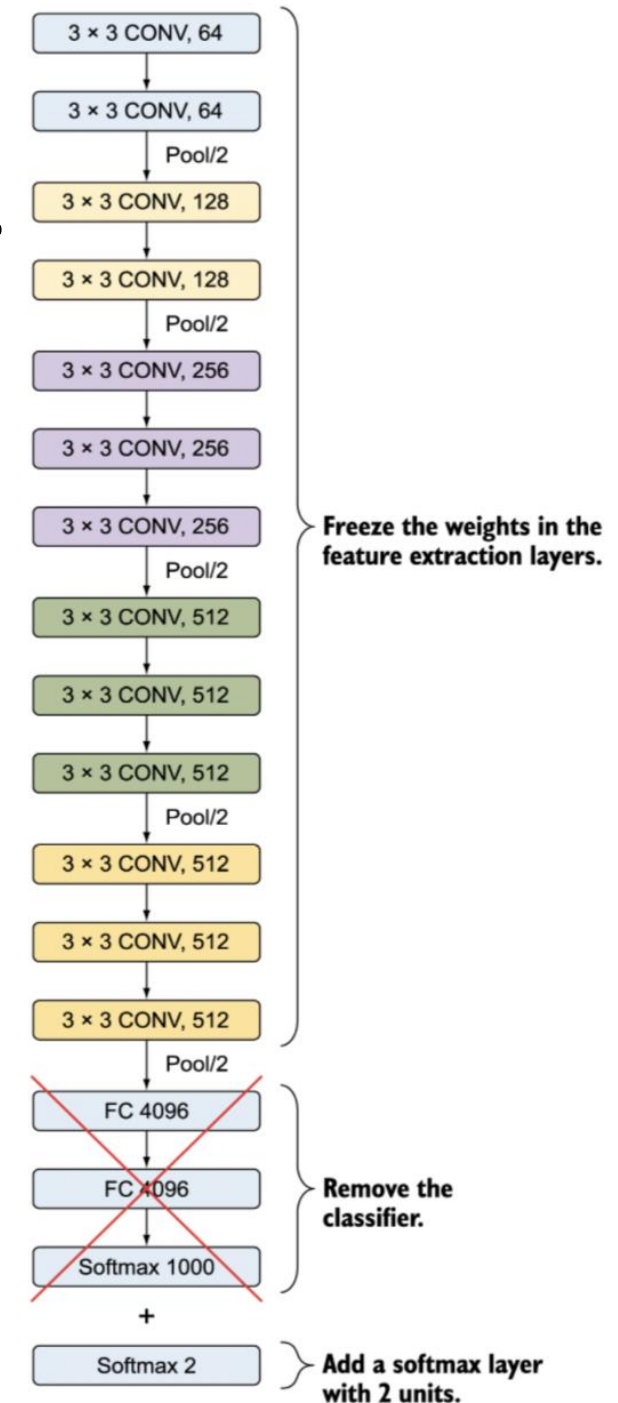


Transfer Learning Approaches

- Pretrained Network as a classifier
 - Doesn't include layers freezing or model training on additional dataset
 - Used directly to classify new images
 - The target domain is same as that of the source domain of the dataset for the pretrained model
 - Using pretrained models in deployment as is
 - E.g. VGG16 can be directly used to classify German Shepherd since the model has seen a number of dog breeds

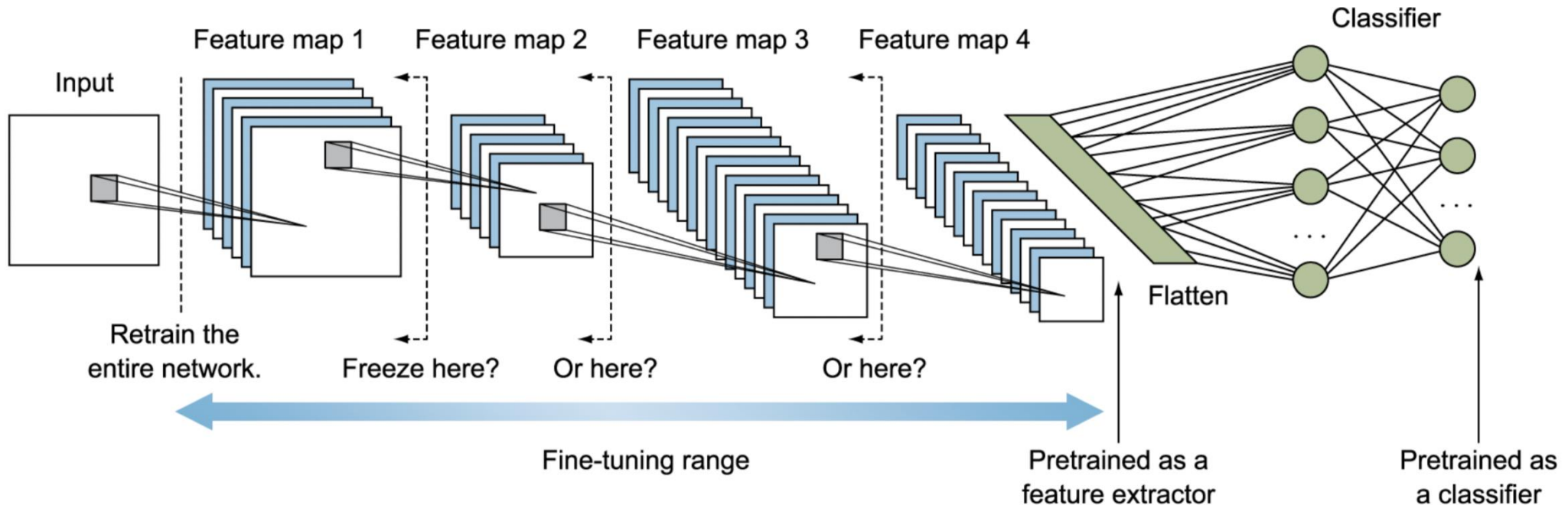
Transfer Learning Approaches - cont.

- Pretrained Network as a feature extractor
 - Freezes the feature extraction layers and modifies/updates the FC and classification layers
 - Used when the new problem domain is similar to the pretrained model dataset domain
 - Allows to reuse the previously learned features



Transfer Learning Approaches - cont.

- Pretrained Network for fine-tuning
 - Freezes a few of the network layers



Fine-tuning

- Fine-tuning is better than training from scratch
 - Helps model to converge faster than starting with randomly initialized weight values
 - Uses already optimized weights to get started with
- Uses smaller learning rate to avoid distortion of the already optimized weights too quickly and too much



Choosing transfer learning levels

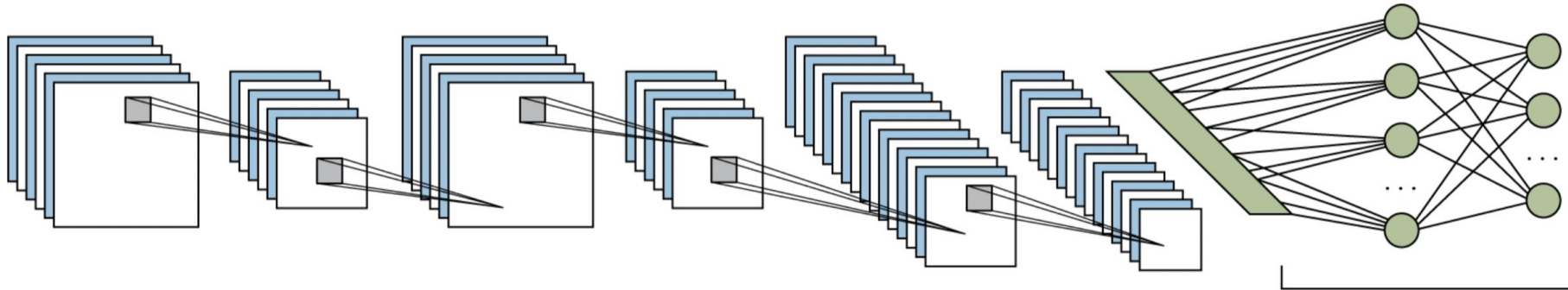
- Choosing the appropriate level of transfer learning is a function of
 - Size of the target dataset (small or large), and
 - Domain similarity of the source and target datasets
- Major scenarios based on the dataset size and the dataset similarity
 - The target dataset is small and similar to the source dataset.
 - The target dataset is large and similar to the source dataset.
 - The target dataset is small and very different from the source dataset.
 - The target dataset is large and very different from the source dataset.

Choosing transfer learning levels

Table 6.1 Transfer learning scenarios

Scenario	Size of the target data	Similarity of the original and new datasets	Approach
1	Small	Similar	Pretrained network as a feature extractor
2	Large	Similar	Fine-tune through the full network
3	Small	Very different	Fine-tune from activations earlier in the network
4	Large	Very different	Fine-tune through the entire network

Choosing transfer learning levels



Scenario #1: You have a small dataset that is similar to the source dataset.

Scenario #2: You have a large dataset that is similar to the source dataset.

Scenario #3: You have a small dataset that is different from the source dataset.

Scenario #4: You have a large dataset that is different from the source dataset.

Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020